

Sincronização

1. Problemas de Concorrência

- Ocorrem quando **vários processos ou threads** acedem **simultaneamente a dados partilhados**.
- Em sistemas **preemptivos**, um processo pode ser interrompido a qualquer momento.
- Sem controlo, o resultado da execução é **indeterminado**.

2. Secção Crítica

Definição: Zona de código que só pode ser executada por um processo/thread de cada vez.

- Normalmente envolve:
 - leitura
 - modificação
 - escrita de dados partilhados

Requisitos de uma solução correta

1. **Exclusão mútua:** Se um processo está na SC, nenhum outro pode estar.
2. **Progresso:** Se ninguém está na SC, um processo à espera deve poder entrar.
3. **Espera limitada:** Um processo não pode esperar indefinidamente para entrar na SC.

3. Mutex Locks

Conceito: Mutex (Mutual Exclusion Lock) garante exclusão mútua.

- Variável inteira controlada pelo SO:
 - **1** → livre
 - **0** → ocupado
- Tem uma **fila de espera** para processos bloqueados.

Operações

- **acquire()**
 - Se mutex = 1 → passa a 0 e entra na SC
 - Se mutex = 0 → processo bloqueia
- **release()**
 - Liberta o mutex
 - Acorda um processo da fila (se existir)

Características

- Simples e eficiente
- Apenas **1 processo** pode aceder ao recurso
- Usado quando existe **um único recurso partilhado**

4. Semáforos

Conceito Geral

- Um **semáforo** é uma variável inteira ≥ 0 , gerida pelo SO.
- Usado para:
 - exclusão mútua
 - sincronização entre processos
 - controlo de múltiplos recursos

Tipos de Semáforos

-Semáforo Binário

- Valores: **0** ou **1**
- Funciona como um mutex
- Usado para exclusão mútua

-Semáforo Contador

- Valores: **0 ... N**
- Controla acesso a **N instâncias de um recurso**
- Ex.: 6 ligações, 6 impressoras, etc.

Operações

- **acquire()** (wait)
 - Se valor > 0 → decrementa | Se valor = 0 → bloqueia

- `release()` (signal)
 - Incrementa
 - Desbloqueia um processo à espera (se existir)

5. Semáforos vs Mutex

Característica	Mutex	Semáforo
Exclusão mútua	✓	✓
Contador > 1	✗	✓
Dono do lock	Sim	Não
Sincronização entre processos	Limitada	Muito flexível

6. Semáforos de Bloqueio

- Usados para **ordenar a execução** de processos.
- Um processo só avança **depois de outro terminar uma etapa**.

Exemplo:

- Processo B só executa depois do Processo A
- Semáforo inicializado a 0
- A faz `release()`, B faz `acquire()`

7. Problemas Clássicos de Sincronização

Produtor–Consumidor

- Produtor coloca itens num buffer
- Consumidor retira itens
- Regras:
 - produtor só escreve se houver espaço
 - consumidor só lê se houver itens
- Usa:
 - semáforo `empty`
 - semáforo `full`
 - mutex para o buffer

Leitores–Escritores

- Vários leitores podem ler ao mesmo tempo
- Apenas **um escritor** pode escrever
- Escritor tem acesso exclusivo

Jantar dos Filósofos

- 5 filósofos, 5 paus
- Cada filósofo precisa de 2 paus para comer
- Problemas possíveis:
 - **deadlock**
 - **starvation**
- Serve para estudar limitações das soluções de sincronização

Gestão de Memória

1. Objetivo da Gestão de Memória

- Garantir que **cada processo tem o seu próprio espaço de endereçamento**.
- Assegurar:
 - **Proteção** entre processos
 - **Eficiência** no uso da memória
 - **Facilidade de execução** dos programas
- A gestão é feita pelo **Sistema Operativo com apoio da MMU**.

2. Alocação Contígua

- Cada processo ocupa **um único bloco contínuo de memória física**.
- Simples de implementar, mas apresenta problemas:
 - **Fragmentação externa**
 - Dificuldade em aumentar o tamanho de um processo

- Algoritmos:
 - First-fit
 - Best-fit
 - Worst-fit

3. Segmentação

Conceito

- O espaço de endereçamento lógico de um processo é dividido em **segmentos**.
- Cada segmento representa uma parte lógica do programa:
 - Código
 - Dados
 - Pilha (stack)
 - Bibliotecas

Endereçamento

- Endereço lógico = $\langle \text{segmento}, \text{deslocamento} \rangle$
- Cada segmento tem:
 - **Base**: endereço inicial em memória física
 - **Limite**: tamanho do segmento

Tradução de Endereços

1. Verifica se o deslocamento $<$ limite
2. Endereço físico = base + deslocamento
3. Caso contrário $\rightarrow$ erro de acesso

Vantagens

- Organização lógica do programa
- Facilita partilha (ex.: código partilhado)

Desvantagens

- **Fragmentação externa**
- Gestão mais complexa

4. Paginação

Conceito

- A memória é dividida em blocos de tamanho fixo:
 - **Páginas** (memória lógica)
 - **Frames** (memória física)
- Elimina a fragmentação externa.

Funcionamento

- O SO mantém uma **tabela de páginas** por processo.
- Cada entrada indica:
 - Frame correspondente
 - Bits de proteção (válido, leitura, escrita, etc.)

Endereçamento

- Endereço lógico = $\langle \text{n}^\circ \text{ da página}, \text{offset} \rangle$
- Tradução:
 1. N° da página indexa a tabela de páginas
 2. Obtém-se o frame
 3. Endereço físico = frame + offset

Vantagens

- Não há fragmentação externa
- Facilita memória virtual
- Permite páginas partilhadas

Desvantagens

- **Fragmentação interna**
- Overhead da tabela de páginas

5. Segmentação vs Paginação (comparação típica de exame)

Segmentação	Paginação
Blocos de tamanho variável	Blocos de tamanho fixo
Visão lógica do programa	Visão física da memória
Fragmentação externa	Fragmentação interna
Endereço (segmento, offset)	Endereço (página, offset)
Mais próxima do programador	Mais próxima do hardware

Memória Virtual

1. Conceito de Memória Virtual

- A memória virtual permite que o espaço de endereçamento lógico de um processo seja maior do que a memória física disponível.
- Apenas parte do processo precisa de estar em memória para executar.
- A gestão é feita pelo Sistema Operativo com apoio da MMU.

Ideias-chave

- Separação entre:
 - **Memória lógica (virtual)** → vista pelos processos
 - **Memória física (RAM)** → real
- Permite:
 - Mais processos em execução
 - Melhor utilização da memória
 - Programas maiores que a RAM

2. Paginação a Pedido (Demand Paging)

- Técnica mais usada para implementar memória virtual.
- As páginas **só são carregadas para a memória quando são realmente necessárias**.

Vantagens

- Menor utilização de memória
- Menor número de operações de I/O
- Melhor tempo de resposta
- Maior grau de multiprogramação

3. Estados de uma Página

Quando um processo acede a uma página, podem ocorrer três situações:

1. **Página válida e presente**
 - Página está na memória
 - Acesso normal
2. **Referência inválida**
 - Página não pertence ao processo
 - Processo é terminado (erro)
3. **Página válida mas não presente**
 - Página está no disco
 - Ocorre uma **falta de página (page fault)**

4. Falta de Página (Page Fault)

- Interrupção gerada quando um processo tenta aceder a uma página que **não está na memória física**.

Tratamento de uma Falta de Página

1. O hardware deteta a falta de página e gera uma exceção
2. O Sistema Operativo ganha o controlo
3. Verifica se a referência é válida:
 - Se inválida → termina o processo
4. Escolhe uma frame livre (ou prepara substituição)
5. Carrega a página do disco para a memória
6. Atualiza a tabela de páginas (bit válido)
7. Reinicia a instrução que causou a falta

O processo **não recomeça do início**, continua exatamente onde parou.

5. Bit Válido–Inválido

- Cada entrada da tabela de páginas tem um **bit de validade**:
 - **v** (válido) → página em memória
 - **i** (inválido) → página não presente
- Inicialmente, todas as páginas estão marcadas como inválidas.
- O bit muda para válido quando a página é carregada.

6. Lazy Swapping

- Também chamado de **paginação a pedido**.
- Só se traz uma página para a memória **quando é necessária**.
- O componente do SO responsável chama-se **pager**.

Interface de Sistema de Ficheiros

1. Conceito de Ficheiro

- Um **ficheiro** é um **tipo de dados abstrato** usado para armazenar informação.
- Pode conter:
 - Programas
 - Dados
 - Texto ou binário
- Para o utilizador, o ficheiro é visto como uma **sequência lógica de dados**.

Tipos de ficheiros

- Executáveis
- Código fonte
- Bibliotecas
- Documentos
- Multimédia
- Arquivos, etc.

2. Atributos de um Ficheiro

Cada ficheiro possui metadados, como:

- **Nome** (legível pelo utilizador)
- **Identificador** (interno do SO)
- **Tipo**
- **Localização** no disco
- **Tamanho**
- **Proteção** (permissões)
- **Datas** (criação, modificação, acesso)

3. Operações sobre Ficheiros

- **create** – criar ficheiro
- **open** – abrir ficheiro
- **read** – ler
- **write** – escrever
- **reposition** – mover ponteiro
- **close** – fechar
- **delete** – remover

-O **open** carrega os atributos do ficheiro para memória.

-O **close** grava alterações e remove a informação da memória.

4. Ficheiros Abertos

- O SO mantém **tabelas de ficheiros abertos**:
 - **Tabela por processo**
 - Ponteiro para a posição atual no ficheiro
 - Permissões
 - **Tabela global**
 - Localização no disco
 - Tamanho
 - Contador de processos que têm o ficheiro aberto

5. Métodos de Acesso a Ficheiros

- **Sequencial:** Leitura/escrita em ordem
- **Direto (aleatório):** Acesso a um bloco específico
- **Indexado:** Usa um índice para localizar blocos rapidamente

6. Estrutura de Diretórios

- Diretório = coleção de ficheiros e subdiretórios.
- Funções:
 - Organizar ficheiros
 - Evitar conflitos de nomes
- Estruturas:
 - Um nível
 - Dois níveis
 - Árvore
 - Grafos (com links)

7. Partilha e Proteção

- Sistemas multiutilizador exigem **partilha controlada**.
- Proteção baseada em:
 - Utilizador
 - Grupo
- Permissões típicas:
 - Leitura
 - Escrita
 - Execução

Implementação de Sistema de Ficheiros

1. Estrutura do Sistema de Ficheiros

O sistema de ficheiros é organizado em **camadas**:

1. Interface do utilizador
2. Sistema de ficheiros lógico
3. Organização de ficheiros
4. Gestão de espaço livre
5. Sistema de ficheiros básico
6. Controlo de E/S (drivers)
7. Hardware

2. FCB – File Control Block

- Estrutura de dados que contém toda a informação de um ficheiro:
 - Permissões
 - Dono e grupo
 - Tamanho
 - Datas
 - Apontadores para blocos de dados
- Em Unix, o FCB corresponde ao **inode**.

3. Implementação das Operações

Create

- Reserva um novo FCB
- Atualiza o diretório

Open

- Verifica se o ficheiro já está aberto
- Atualiza:
 - Tabela global
 - Tabela do processo

Read / Write

- Feitas através de um **descritor de ficheiro**
- Usam o ponteiro associado ao processo

Close

- Remove entrada da tabela do processo
- Decrementa contador global
- Se ninguém mais usar → remove entrada global

4. Implementação de Diretórios

- Diretório associa:
 - Nome do ficheiro → FCB
- Implementações possíveis:
 - Lista simples
 - Lista ordenada
 - Tabela de hash (mais eficiente)

5. Métodos de Reserva de Espaço em Disco

Reserva Contígua

- Blocos consecutivos
- Rápido
- Problemas:
 - Fragmentação externa
 - Ficheiro não cresce facilmente

Lista Ligada

- Blocos espalhados
- Fácil de crescer
- Acesso aleatório lento

FAT (File Allocation Table)

- Variante da lista ligada
- Tabela em memória melhora desempenho

Reserva Indexada

- Tabela de índices com apontadores
- Acesso direto eficiente
- Usado em sistemas modernos (ex.: Unix)

6. Gestão de Espaço Livre

- **Bitmap (vetor de bits)**
 - 0 → livre
 - 1 → ocupado
- **Listas encadeadas**
- Técnicas para melhorar desempenho:
 - Cache de disco
 - Prefetching
 - Buffers

7. Recuperação

- Falhas podem causar inconsistência.
- Soluções:
 - Verificação (fsck, chkdsk)
 - Journaling (sistemas transacionais)
 - Backups (completos e incrementais)

Armazenamento

1. Objetivo do Armazenamento

- O armazenamento fornece **memória secundária** para:
 - Guardar dados de forma persistente
 - Suportar sistemas de ficheiros
 - Permitir memória virtual
- É **mais lento que a RAM**, mas **não volátil**.

2. Tipos de Dispositivos de Armazenamento

Discos Magnéticos

- **Discos rígidos (HDD):**
 - Dados armazenados em pratos giratórios
 - Cabeças de leitura/escrita
- **Fitas magnéticas:**
 - Muito usadas para backups
 - Acesso sequencial
 - Baixo custo por GB

Discos SSD

- **Solid State Drives**
- Sem partes mecânicas
- Muito rápidos em acessos aleatórios
- Mais caros por GB

3. Conceitos Importantes

- **Tempo de acesso ao disco:**
 - Tempo de busca (seek time)
 - Latência rotacional
- **Taxa de transferência:**
 - Velocidade de leitura/escrita
- **Bloco:**
 - Unidade mínima de transferência usada pelo SO
 - Tipicamente 4 KB

4. Estrutura de Disco

- Disco visto como um **vetor linear de blocos lógicos**
- Blocos lógicos são mapeados para setores físicos
- O SO **não lida diretamente com setores**, mas com blocos

5. Ligação de Discos a Computadores

- Interfaces comuns:
 - SATA
 - USB
 - NVMe (via PCIe)
- Sistemas de rede:
 - **NAS** – acesso a ficheiros pela rede
 - **SAN** – acesso a blocos remotos (parecem discos locais)

6. Escalonamento de Disco

Objetivo: **reduzir o tempo médio de acesso**

Algoritmos

- **FCFS** – simples, pouco eficiente
- **SSTF** – menor tempo de busca (pode causar starvation)
- **SCAN / C-SCAN**
- **LOOK / C-LOOK**

-LOOK e SSTF são geralmente os melhores.

7. Gestão de Disco

- **Formatação:**
 - Física (baixo nível)
 - Lógica (criação do sistema de ficheiros)
- **Boot:**
 - ROM → MBR → boot sector → kernel
- **Blocos defeituosos:**
 - Tratados com sector sparing

8. RAID

- **RAID** = Redundant Array of Independent Disks
- Objetivos:
 - Melhor desempenho
 - Maior fiabilidade
- Níveis:
 - RAID 0 – desempenho
 - RAID 1 – espelhamento
 - RAID 5/6 – paridade
 - RAID 10 – desempenho + redundância

Entradas e Saídas (E/S)

1. Gestão de E/S

- O SO gere uma **grande variedade de dispositivos**:
 - Discos
 - Teclado
 - Rato
 - Impressoras
 - Rede
- Apesar das diferenças, o SO fornece uma **interface uniforme**.

2. Hardware de E/S

Componentes comuns:

- Portos
- Barramentos (bus)
- Controladores (adaptadores)
- Endereços de dispositivos
- Instruções de E/S

3. Métodos de Comunicação com Dispositivos

Polling

- CPU verifica repetidamente o estado do dispositivo
- Simples, mas ineficiente

Interrupções

- Dispositivo avisa a CPU quando precisa de atenção
- Mais eficiente que polling

DMA (Direct Memory Access)

- Controlador transfere dados diretamente entre memória e dispositivo
- CPU quase não intervém
- Muito eficiente para grandes volumes de dados

4. Device Drivers

- Software que controla dispositivos específicos
- Funções:
 - Esconder detalhes do hardware
 - Oferecer interface padronizada
- Cada dispositivo tem o seu driver

5. Tipos de Operações de E/S

- **Bloqueantes**
 - Processo espera até terminar a operação
- **Não bloqueantes**
 - Retorna imediatamente
- **Assíncronas**
 - Processo continua
 - É notificado quando termina

6. Subsistema de E/S do Núcleo

Responsável por:

- Escalonamento de pedidos
- Buffers
- Cache
- Spooling (ex.: impressoras)
- Proteção de dispositivos
- Gestão de erros

7. Proteção e E/S

- Todas as instruções de E/S são **privilegiadas**
- Processos não acedem diretamente ao hardware
- O SO valida todos os pedidos de E/S

8. Desempenho em E/S

- E/S tem grande impacto no desempenho global
- Problemas:
 - Cópias de dados
 - Interrupções frequentes
- Soluções:
 - DMA
 - Buffers e cache
 - Reduzir trocas de contexto

Proteção

1. Conceito de Proteção

- **Proteção** garante que os **recursos do sistema** são:
 - Acedidos corretamente
 - Apenas por quem tem autorização
- Um **Sistema Operativo** pode ser visto como uma **coleção de objetos**:
 - Ficheiros, memória, CPU, dispositivos, processos
- Cada objeto tem:
 - Um nome
 - Um conjunto de operações permitidas

Princípio do Mínimo Privilégio

- Utilizadores e programas recebem **apenas os privilégios estritamente necessários**.
- Reduz erros e impactos de ataques.

2. Domínios de Proteção

- Um **domínio de proteção** é um **conjunto de permissões**.
- Pode representar:
 - Um utilizador
 - Um processo
 - Um procedimento

Exemplos:

- Modo utilizador vs modo núcleo
- Em Unix:
 - Utilizador
 - Grupo
 - Outros

Troca de Domínio

- Um processo pode mudar de domínio:
 - Ex.: execução de um programa com **setuid**
 - Processo passa a ter permissões do dono do ficheiro

3. Direitos de Acesso

- Direito de acesso = $\langle \text{objeto}, \text{conjunto de operações} \rangle$
- Exemplos de operações:

- Ler
- Escrever
- Executar
- Apagar
- Modificar permissões

4. Matriz de Acessos

- Proteção pode ser representada como uma **matriz**:
 - Linhas → domínios
 - Colunas → objetos
 - Célula → operações permitidas

Utilização

- Um acesso só é permitido se estiver definido na matriz.
- Separação importante:
 - **Mecanismo** (como controlar)
 - **Política** (quem pode aceder)

5. Implementações da Matriz

ACL – Access Control Lists

- Lista de permissões por objeto
- Exemplo:
 - João pode ler
 - Maria pode escrever
- Usada em Unix e Windows

Capacidades

- Lista de permissões por domínio
- O processo apresenta a sua capacidade para aceder ao recurso

6. Controlo de Acesso Baseado em Papéis

- Utilizadores têm **papéis (roles)**.
- Cada papel define permissões.
- Facilita a gestão em sistemas grandes.

Segurança

1. Diferença entre Proteção e Segurança

- **Proteção**: controlo interno do acesso aos recursos
- **Segurança**: defesa contra ameaças externas e internas
 - Ataques
 - Intrusões
 - Roubo de informação

2. Segurança de Recursos

Perímetro de Segurança

- Protege uma zona da rede.
- Exemplo típico: **Firewall**
- Controla:
 - Serviços
 - Direção do tráfego
 - Utilizadores
 - Conteúdos

3. Controlo de Acesso (AAA)

- **Autenticação** – quem és
- **Autorização** – o que podes fazer
- **Auditoria** – o que fizeste

Métodos de Autenticação

- Algo que sabes (password)
- Algo que tens (cartão)
- Algo que és (biometria)

4. Segurança de Comunicação

- Objetivo: **comunicação segura**
- Técnicas:
 - HTTPS
 - VPN
 - SSH
 - Secure Mail
- Criação de um canal seguro antes da troca de dados

5. Segurança da Informação

Baseia-se em três pilares:

Confidencialidade: Apenas utilizadores autorizados podem ler a informação.

Integridade: A informação não pode ser alterada indevidamente. Detetar alterações não autorizadas.

Autenticidade: Garantir a identidade do emissor da informação.

6. Criptografia

Conceito

- Protege dados através de:
 - Chaves, Algoritmos, Cifra e decifra

Tipos

- **Criptografia Simétrica:** Mesma chave para cifrar e decifrar
- **Criptografia Assimétrica:** Chave pública + chave privada

7. Assinatura Digital

- Garante:
 - **Integridade**
 - **Autenticidade**
- Não garante confidencialidade.
- Processo:
 - Emissor assina com chave privada
 - Receptor verifica com chave pública

8. Certificados Digitais

- Garantem que uma chave pública é legítima.
- Emitidos por uma **entidade certificadora**.
- Certificado é assinado digitalmente pela entidade.

Máquinas Virtuais

1. Conceito de Máquinas Virtuais (VMs)

- Uma **Máquina Virtual (VM)** é uma abstração que permite executar vários ambientes de execução independentes numa única máquina física.
- Cada VM comporta-se como um computador completo, podendo executar:
 - Aplicações
 - Sistemas Operativos diferentes

-Objetivo principal: abstrair o hardware físico.

2. Componentes Principais

- **Host**
 - Hardware físico
 - Eventualmente um sistema operativo base
- **VMM (Virtual Machine Monitor) / Hypervisor**
 - Cria, gere e executa máquinas virtuais
 - Fornece uma interface semelhante ao hardware real

- **Guest**
 - Sistema operativo e aplicações a correr dentro da VM

3. Funcionamento Geral

- O VMM intercepta operações sensíveis do guest.
- Garante:
 - Isolamento entre VMs
 - Proteção do sistema host
- Cada VM tem:
 - CPU virtual
 - Memória virtual
 - Dispositivos virtuais

4. Vantagens das Máquinas Virtuais

- **Isolamento e segurança:** Falhas ou ataques numa VM não afetam as outras
- **Execução simultânea:** Vários SOs na mesma máquina
- **Compatibilidade:** Executar software antigo ou específico
- **Desenvolvimento e testes:** Ideal para testar SOs e aplicações
- **Snapshots:** Guardar e restaurar estados da VM
- **Clonagem e migração:** Copiar ou mover VMs facilmente

5. Tipos de Hypervisores

Tipo 0 (Firmware)

- Virtualização ao nível do hardware
- Ex.: soluções empresariais dedicadas

Tipo 1 (Bare-metal)

- Corre diretamente sobre o hardware
- Alto desempenho
- Usado em servidores

Tipo 2 (Hosted)

- Corre sobre um sistema operativo normal
- Mais simples de usar
- Usado em desktops e portáteis

6. Virtualização vs Contentores

- **Máquinas Virtuais**
 - Virtualizam hardware
 - Cada VM tem o seu próprio SO
 - Mais pesadas
- **Contentores**
 - Partilham o kernel do SO
 - Mais leves e rápidos
 - Menor isolamento que VMs